Preference-Conditioned Deep Reinforcement Learning with Risk-Augmented GP-Evolved Actions for Dynamic Workflow Scheduling in Cloud–Edge Computing

Anonymous Authors

***Abstract*—Online workflow scheduling in heterogeneous Cloud–Edge systems must allocate resources before future workflow arrivals are known. Resource choices have state-dependent effects on both workflow tardiness and energy consumption. Edge Nodes avoid Cloud provisioning and reduce wide-area communication delays but have limited capacity, whereas Cloud Servers provide elastic and powerful computing capacity but incur additional provisioning and communication overheads. Their relative benefits further depend on task characteristics, workflow dependencies, and energy efficiency. Scheduling policies must therefore adapt to user-requested objective trade-offs and evolving system states. Genetic programming (GP) hyper-heuristics have demonstrated effectiveness in evolving resource-selection rules for dynamic workflow scheduling, but usually apply them as fixed policies. Deep reinforcement learning enables adaptive control, yet directly treating a variable set of feasible resources as actions while handling multiple preferences complicates policy learning. We propose a preference-conditioned deep reinforcement learning framework with risk-augmented GP-evolved actions, termed PRS-GPDRL. Cooperative coevolution genetic programming first evolves coupled task- and resource-selection rules. Quality- and behaviour-aware filtering then retains a compact resource-selection rule pool from the final CCGP archive. Each retained rule is augmented with base and preference-conditioned risk modes. A preference-conditioned multi-objective double deep Q-network then selects a rule-mode action at each allocation decision. Across 12 scenarios and 30 independent runs, PRS-GPDRL achieves the best average ranks for both HV and IGD and is statistically better than or comparable to CCGP in 11 scenarios for each metric.**

***Index Terms*—Cloud–Edge computing, dynamic workflow scheduling, genetic programming, deep reinforcement learning, multi-objective optimisation.**

I. Introduction

The continuing growth of digital services is increasing demand for computing capacity both in centralised Cloud data centres and at the network Edge. IDC estimated worldwide spending on Edge computing at nearly USD 261 billion in 2025 and projected it to reach USD 380 billion by 2028 [1]. At the same time, the energy footprint of computing infrastructure is becoming substantial. The International Energy Agency reports that data centres consumed approximately 415 TWh of electricity in 2024 and projects this consumption to more than double to around 945 TWh by 2030 [2]. These trends make the timely and energy-efficient use of distributed computing capacity an increasingly important systems problem.

Cloud–Edge computing addresses this demand by combining latency-proximate Edge Nodes with elastic and powerful Cloud Servers. It provides a heterogeneous execution environment for workflows in scientific computing, data analytics, and Internet of Things services [3], [4], [5]. Such applications are commonly represented as directed acyclic graphs (DAGs), in which tasks are linked by precedence and data dependencies. In an online system, workflows arrive dynamically and their future information is unavailable. The scheduler must therefore order ready tasks and allocate resources from the currently observed workflow and system states.

Container-based virtualisation provides the execution substrate for these workflows. It packages each task with its software dependencies into a lightweight execution unit that can be deployed across heterogeneous Edge Nodes and Cloud Servers [6]. Modern container platforms also permit CPU resources to be adjusted after deployment. Kubernetes supports in-place resizing of CPU requests and limits for running Pods, while Docker can update container CPU shares and quotas through cgroup-backed controls [7], [8], [9]. These capabilities provide practical support for concurrent container execution with runtime CPU reallocation. In the considered system, co-located task containers share the CPU capacity of their resource, and their allocations are updated as tasks start or complete. Consequently, a new placement may alter the execution rates of running tasks and propagate its effects through workflow dependencies.

These interactions make a placement consequential beyond the task being assigned. It changes the capacity available to co-located tasks, the release times of successors, and the period over which resources remain active. Its tardiness and energy effects therefore unfold over subsequent events rather than being determined by the immediate execution time alone. Moreover, a placement that is favourable under one workload state or objective weighting may be undesirable under another. Since weighted tardiness and energy must be considered jointly [10], [11], online scheduling requires a policy that can vary its resource-selection behaviour with both the observed state and the requested tardiness–energy preference.

Genetic programming hyper-heuristics (GPHHs) are attractive for dynamic scheduling because they evolve executable priority rules offline and apply them quickly online. They have been developed from single-tree Cloud workflow rules to multi-objective rule sets and interacting multi-tree or dual-

tree rules [12], [13], [14], [15], [16]. Cooperative coevolution genetic programming (CCGP) further decomposes coupled task-selection and resource-management rules into cooperating subpopulations [17]. However, once an evolved heuristic is deployed, its component rules are normally fixed throughout a scheduling run. Different rules may be effective for different system states or objective preferences, but a fixed heuristic cannot learn when to switch among their complementary behaviours.

Deep reinforcement learning (DRL) provides state-dependent control, but direct resource-selection actions are difficult to represent because the set of feasible resources changes over time [10], [18], [19]. Rule-assisted DRL avoids this difficulty by selecting a scheduling rule from a fixed action set. This principle has been explored with manually designed or GP-evolved rules in dynamic job-shop scheduling [20], [21], [22]. For dynamic Fog workflow scheduling, He et al. use GP to construct a diverse action set and DRL to select evolved actions online, but learn a single-objective tardiness policy [23]. These studies improve the quality and behavioural diversity of rule actions, but their online policies target a single scheduling objective. Preference-conditioned multi-objective reinforcement learning (MORL) shows that one policy can respond to different objective weights [24], [25], [26], [27], while PSLGP embeds preferences into a GP heuristic for multi-objective dynamic job-shop scheduling [28]. Consequently, accommodating changing user priorities within one online scheduler remains an open issue for dynamic Cloud–Edge workflows.

To address this gap, we propose PRS-GPDRL, a preference-conditioned deep reinforcement learning framework with risk-augmented GP-evolved actions. CCGP first evolves coupled tree-based task- and resource-selection rules. A quality- and behaviour-aware extraction procedure then retains a compact pool of complementary resource-selection rules from the final CCGP archive. Pairing this rule pool with the base and preference-conditioned risk modes gives a fixed-dimensional rule-mode action space. Finally, a preference-conditioned multi-objective double deep Q-network (DDQN) learns when to select each rule-mode action, while a fixed upward-rank rule orders ready tasks. Evolutionary search discovers interpretable resource-selection rules, while the DDQN selects among them according to the current system state and requested objective trade-off.

The main contributions are as follows:

- We propose a CCGP-to-DRL scheduling framework that evolves coupled GP rules offline and extracts complementary resource-selection rules into a compact pool. Pairing this pool with two modes creates a fixed rule-mode action space for each trained agent while separating the variable candidate-resource set from the DDQN output.
- We introduce risk-stratified action augmentation to complement rule-level GP selection with local, preference-conditioned resource-priority adjustment. A preference-conditioned multi-objective DDQN combines this action design with FiLM-based preference conditioning, vector-valued action costs, and augmented weighted Tchebycheff scalarisation, enabling one network to represent different tardiness–energy trade-offs.
- Across 12 dynamic scheduling scenarios and 30 independent runs, PRS-GPDRL obtains the best average ranks for both HV (1.403) and IGD (1.525). It is statistically better than or comparable to CCGP in 11 of 12 scenarios and to GATES and LWFF in at least 11 of 12 scenarios for each metric.

II. Related Work

A. Dynamic Workflow Scheduling in Cloud–Edge Systems

Research on dynamic Cloud–Edge workflow scheduling can be distinguished by where its decision structure originates. Constructive schedulers encode domain knowledge through task ranks, sub-deadlines, resource reuse, incremental costs, or non-dominated candidate construction. These mechanisms support deadline, energy, and monetary objectives without policy training and retain low online decision cost [29], [30], [31], [32], [33]. Their behaviour, however, is governed by manually specified scoring and selection structures, which restricts how scheduling attributes can be recombined as system states and objective preferences change.

Learning-based schedulers move this combination into a learned policy. DRL and multi-agent DRL have consequently been applied to time–energy scheduling, multi-objective resource allocation, and collaborative workflow offloading across the Cloud–Edge continuum [10], [11], [18]. GATES further uses heterogeneous-graph attention to represent varying ready-task and virtual-machine sets, with an evolution strategy learning the allocation policy [19]. GP hyper-heuristics take a different route: they search directly for executable priority rules rather than a neural allocation mapping. In this line, CCGP coevolves task-selection, resource-selection, and container-scaling trees and returns non-dominated scheduling heuristics for dynamic Cloud–Fog workflows [17]. The resulting heuristics offer diverse trade-offs, but each applies a fixed combination of component rules during execution; they do not provide state-dependent switching among complementary resource rules under a continuously varying bi-objective preference.

B. Rule-Assisted Reinforcement Learning for Dynamic Scheduling

Rule-assisted DRL separates high-level adaptation from the concrete scheduling decision. Instead of exposing a variable set of jobs or resources as actions, the agent selects a fixed-dimensional rule that subsequently ranks the feasible candidates. Liu et al. establish this indirect action representation using manually designed dispatching rules in dynamic flexible job shops [20]. Although the representation accommodates changing candidate sets, the manual rule library places an inherent limit on the quality and behavioural diversity available to the agent [21].

Subsequent research replaces the manual library with evolved rules. Niching and behaviour clustering preserve distinct scheduling behaviours, while GP supplies executable routing or sequencing actions for online DRL selection [21], [22]. The same evolutionary action principle has also been

applied to tardiness minimisation in dynamic Fog workflow scheduling [23]. These studies broaden the source of rule actions through evolution, but primarily address single-objective scheduling or optimise for a fixed objective setting; continuously changing tardiness–energy preferences are outside their scope.

C. Preference-Conditioned Multi-Objective Learning

Multi-objective reinforcement learning (MORL) addresses sequential decisions with vector-valued outcomes by representing more than one objective trade-off [24], [25]. One line learns explicit policy sets or a continuous Pareto manifold, providing coverage across non-dominated behaviours [34], [35]. A second line conditions a shared value function or policy on the requested preference. Dynamic weights, desired returns, objective-specific action distributions, and preference-controllable meta-policies are different conditioning signals for the same purpose: adapting one learned model without retraining it for every trade-off [26], [27], [36], [37], [38].

Pareto set learning develops a related preference-to-solution view. Augmented Tchebycheff scalarisation can train a model that maps preferences to approximate Pareto solutions [39], and PSLGP carries this principle into preference-conditioned GP heuristics for multi-objective dynamic scheduling [28]. Collectively, these studies establish how a model can respond continuously to changing preferences. Its extension to dynamic Cloud–Edge workflow scheduling has received limited attention.

III. Problem Modelling

We consider a standard distributed and heterogeneous Cloud–Edge computing environment. Let $R = C \cup E$ be the set of heterogeneous computing resources, where C and E are the sets of Cloud Servers and Edge Nodes, respectively. Each resource $r \in R$ is a VM with CPU capacity $\Gamma^c(r)$. Edge Nodes form a fixed resource set with limited computing capacity, whereas Cloud Servers can be provisioned on demand. Tasks are packaged as containers through existing image files, and the containers are deployed to resources. A resource can run multiple containers simultaneously, but their total allocated CPU cannot exceed the CPU capacity of the resource at any time. A container is the minimum execution unit and executes only one task at a time. The scheduling model treats CPU capacity as the adjustable resource.

Let B_{edge} and L_{edge} be the bandwidth and latency for communications within the edge side, respectively. Let B_{cloud} and L_{cloud} be the bandwidth and latency for communications involving Cloud Servers, respectively. Each resource has static power and dynamic power. Static power $p^s(r)$ includes base power $p^b(r)$ and CPU-based idle power $p^{ic}(r)$, where $p^s(r) = p^b(r) + p^{ic}(r)$. Dynamic power is defined as the difference between full power and idle power, including CPU-based power $p^{dc}(r) = p^{fc}(r) - p^{ic}(r)$. The energy consumption of both Cloud Servers and Edge Nodes is considered in this work.

Let $\mathcal{W}$ be the set of workflow applications. Each workflow $\omega \in \mathcal{W}$ contains a set of tasks $A(\omega)$. The arrival time, weight, and deadline of workflow ω are $\rho^a(\omega)$, $\rho^w(\omega)$, and $\rho^d(\omega)$, respectively. Each task $\alpha \in A(\omega)$ is represented by a tuple $\{\phi(\alpha), \tau^l(\alpha), \mu^c(\alpha), A^{pre}(\alpha), A^{suc}(\alpha)\}$, where $\phi(\alpha) \in \{0, 1\}$ denotes the task category, $\tau^l(\alpha)$ is its CPU workload, $\mu^c(\alpha)$ is its minimum CPU requirement, and $A^{pre}(\alpha), A^{suc}(\alpha) \subseteq A(\omega)$ are its predecessor and successor task sets, respectively. $d(\alpha_1, \alpha_2)$ is the data transferred from task α_1 to its successor task $\alpha_2 \in A^{suc}(\alpha_1)$. The execution time of a task depends on its category and the CPU configuration of its container. Tasks fall into the following two categories.

- $\phi(\alpha) = 0$. The container's CPU needs to meet the minimum CPU demand of the task, while improving the container's CPU configuration cannot reduce the execution time of the task. Such tasks are usually CPU-insensitive.
- $\phi(\alpha) = 1$. The container's CPU needs to meet the minimum CPU demand of the task, while improving the container's CPU configuration can reduce the execution time of the task, and vice versa. Such tasks are usually CPU-sensitive.

We formulate the dynamic workflow scheduling problem with workflow deadlines as an optimisation problem aiming to minimise the system-oriented total energy consumption and workflow-oriented total weighted tardiness, subject to the following scheduling and resource constraints:

- No task can be executed until all its predecessor tasks have been completed.
- Each task must be executed by a container with sufficient CPU capacity to satisfy its minimum CPU requirement.
- Each resource can deploy and run multiple containers simultaneously, but the sum of their allocated CPU capacities cannot exceed the resource's CPU capacity at any moment.

We assume scheduling as a discrete-event control problem. Let $\mathcal{T}$ be the set of decision points, and $t \in \mathcal{T}$ be a specific decision point. Decision points typically correspond to moments when the system's state changes, such as when a workflow is submitted, a task becomes ready, a task starts execution, or a task is completed. Let $x^s(\alpha)$ and $x^f(\alpha)$ be the execution start time and execution finish time of task α. Let $y(\alpha) \in R$ be the assigned resource of task α in the schedule. Let $\nu^c(\alpha, t)$ be the size of CPU allocated to the container used to execute task α at time t.

The problem can then be formulated as

$$\min \mathbb{E} = \sum_{r \in R} \int_{\mathcal{T}} P(r, t) dt, \tag{1}$$

$$\min \mathbb{T} = \sum_{\omega \in \mathcal{W}} \rho^w(\omega) \times \max \left\{ 0, \max_{\alpha \in A(\omega)} \left\{ x^f(\alpha) \right\} - \rho^d(\omega) \right\}, \tag{2}$$

$$s.t. : x^s(\alpha) \geq \rho^a(\omega), \ \forall \ \omega \in \mathcal{W}, \alpha \in A(\omega), \tag{3}$$

$$x^f(\alpha_1) + x^c(\alpha_1, \alpha_2) + x^l(\alpha_2) \leq x^s(\alpha_2), \ \forall \alpha_2 \in A^{suc}(\alpha_1), \tag{4}$$

$$\begin{cases} x^f(\alpha) - x^s(\alpha) = \dfrac{\tau^l(\alpha)}{\mu^c(\alpha)}, & \phi(\alpha) = 0, \\ \displaystyle\int_{x^s(\alpha)}^{x^f(\alpha)} \nu^c(\alpha, t) dt = \tau^l(\alpha), & \phi(\alpha) = 1. \end{cases} \tag{5}$$

$$\nu^c(\alpha,t) \geq \mu^c(\alpha),\ \forall t \in \mathcal{T}, \tag{6}$$

$$\sum_{\alpha \in A_r(t)} \nu^c(\alpha,t) \leq \Gamma^c(r),\ \forall r \in R, \forall t \in \mathcal{T}, \tag{7}$$

$$x^l(\alpha) = \begin{cases} L_{\text{edge}}, & y(\alpha) \in E, \\ L_{\text{cloud}}, & y(\alpha) \in C. \end{cases} \tag{8}$$

$$x^c(\alpha_1,\alpha_2) = \begin{cases} 0, & y(\alpha_1) = y(\alpha_2), \\ \dfrac{d(\alpha_1,\alpha_2)}{B_{\text{edge}}}, & y(\alpha_1) \neq y(\alpha_2), \\ & y(\alpha_1), y(\alpha_2) \in E, \\ \dfrac{d(\alpha_1,\alpha_2)}{B_{\text{cloud}}}, & \text{otherwise}. \end{cases} \tag{9}$$

The CPU allocation of containers is updated whenever a task starts or completes on a resource. In this work, resource assignment is determined by the scheduling policy, and CPU allocation is then updated according to the following rule. Let $A_r(t)$ be the set of running tasks on resource r at time t, i.e.,

$$A_r(t) = \{\alpha \mid y(\alpha) = r,\ x^s(\alpha) \leq t < x^f(\alpha)\}. \tag{10}$$

Let $S_r(t)$ be the set of CPU-sensitive running tasks on resource r at time t:

$$S_r(t) = \{\alpha \in A_r(t) \mid \phi(\alpha) = 1\}. \tag{11}$$

The remaining CPU capacity after satisfying the minimum CPU requirements of all running tasks is

$$\Gamma^c_{\text{rem}}(r,t) = \Gamma^c(r) - \sum\nolimits_{\beta \in A_r(t)} \mu^c(\beta). \tag{12}$$

For each running task $\alpha \in A_r(t)$, the CPU allocation is calculated as

$$\nu^c(\alpha,t) = \begin{cases} \mu^c(\alpha), & \phi(\alpha) = 0, \\ \mu^c(\alpha) + \dfrac{\Gamma^c_{\text{rem}}(r,t)}{|S_r(t)|}, & \phi(\alpha) = 1. \end{cases} \tag{13}$$

Thus, CPU-insensitive tasks always use their minimum CPU requirements, while CPU-sensitive tasks equally share the remaining CPU capacity.

The CPU utilisation of resource r at time t is

$$U^c(r,t) = \frac{\sum_{\alpha \in A_r(t)} \nu^c(\alpha,t)}{\Gamma^c(r)}. \tag{14}$$

The instantaneous power consumption of resource r is modelled as

$$P(r,t) = p^s(r) + p^{dc}(r)\left(U^c(r,t)\right)^3. \tag{15}$$

The above equations hold for $\forall\ \omega \in \mathcal{W}$, $\forall\ \alpha, \alpha_1, \alpha_2 \in A(\omega)$, $\forall\ r \in R$, and $\forall t \in \mathcal{T}$ when the corresponding quantities are defined.

Eqs. (1) and (2) are the two considered objectives, respectively. Eq. (1) gives the total energy consumption by integrating the instantaneous power $P(r,t)$ over all Cloud Servers and Edge Nodes. Eq. (2) is the workflow-oriented total weighted tardiness. Constraint (3) ensures that the workflow cannot be executed until it is submitted. Constraint (4) defines workflow precedence. A task can start only after all immediate predecessors have finished and their output data have arrived. Eq. (5) governs the two task categories and ensures that task α completes workload $\tau^l(\alpha)$ during $[x^s(\alpha), x^f(\alpha)]$ under the corresponding CPU allocation. Constraint (6) ensures that a container's CPU allocation satisfies the task's minimum requirement. Constraint (7) ensures that allocated container CPU does not exceed resource capacity at time t. Eq. (8) defines the network latency $x^l(\alpha)$, which depends on whether the task is assigned to an Edge Node or a Cloud Server. Eq. (9) gives the communication time between task α_1 and its successor α_2. Eq. (10) defines the set of running tasks on a resource. Eq. (11) defines the set of CPU-sensitive running tasks on a resource. Eq. (12) calculates the remaining CPU capacity after the minimum requirements of all running tasks have been satisfied. Eq. (13) defines the CPU allocation of containers after task-start or task-completion events. Eq. (14) is the CPU-based resource utilisation. Eq. (15) models the instantaneous power consumption $P(r,t)$ of a resource r at time t. The power consists of the static power $p^s(r)$ and the CPU-based dynamic power, which scales cubically with CPU utilisation. For on-demand Cloud Servers, Eq. (15) is evaluated only over recorded utilisation intervals of provisioned resources. Unprovisioned Cloud Servers do not contribute to the energy integral in Eq. (1).

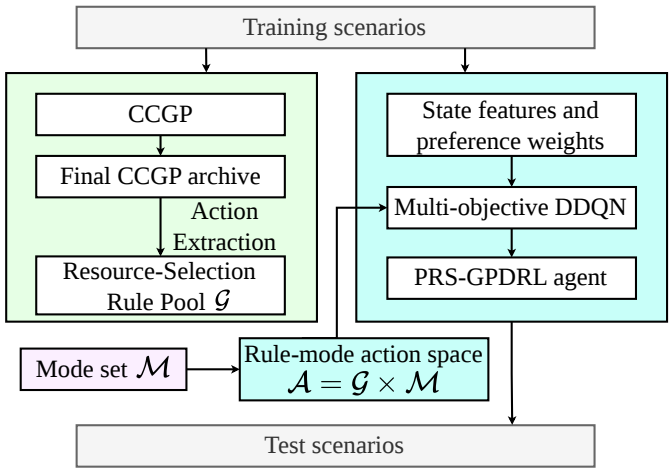

Fig. 1. Overall framework of PRS-GPDRL.

IV. The Proposed PRS-GPDRL Algorithm

A. Overall Framework of PRS-GPDRL

Fig. 1 shows the overall framework, which contains offline training and online scheduling. For each training scenario, CCGP produces the final CCGP archive, from which action extraction retains a compact resource-selection rule pool $\mathcal{G}$. The base and preference-conditioned risk modes form the mode set $\mathcal{M}$; pairing the two sets gives the rule-mode action space $\mathcal{A}$. Using this action space, the state features, and the preference weights, the multi-objective DDQN is trained to form the PRS-GPDRL agent. The formal action-space definition is given in Section IV-D.

During online scheduling, when multiple tasks are simultaneously ready, the fixed upward-rank rule determines their scheduling order [40]. For the task currently being scheduled, the PRS-GPDRL agent first selects a resource-selection rule

and mode; the selected action then scores the feasible resources and assigns the task to the resource with the lowest priority.

B. CCGP Training and Action Extraction

CCGP training. The CCGP stage follows the cooperative coevolution mechanism used for tree-based scheduling rules. CCGP maintains two subpopulations: one for tree-based task-selection rules and the other for tree-based resource-selection rules. During cooperative evaluation, an individual in one subpopulation is paired with a representative from the other. The resulting complete tree-based scheduling heuristic is evaluated using energy consumption and weighted tardiness. During CCGP evolution, evaluated complete scheduling heuristics are recorded in an archive. Until the stopping criterion is met, CCGP updates the subpopulations by elitism, reproduction, crossover, and mutation; during crossover, a subtree from an archive individual replaces a subtree of the parent rule. At the final generation, duplicate solutions are removed from the archive set (AS), and its first non-dominated front is retained as the candidate set for subsequent resource-selection rule extraction.

Rule representation. The tree-based rules are represented by terminal and function nodes. Terminal nodes provide scheduling attributes observed from the current workflow, task, candidate resource, and system state, while function nodes combine these values to compute priority scores. The task- and resource-selection rules use different terminal subsets and share the same function set. Table I summarises the terminals used by the CCGP rules and the corresponding attributes used by the DRL state encoder. The GP and DDQN columns indicate how each attribute is used by the two components. Further details of CCGP training are given in [17].

Action extraction. After CCGP training, PRS-GPDRL extracts a compact pool of resource-selection rules represented as GP trees from the candidate set retained from AS. Action extraction reduces the rule-pool size while preserving diversity, thereby lowering the difficulty of subsequent DRL training. Let g_i denote candidate resource-selection rule i. The extraction procedure constructs $\mathcal{G}$ as follows.

Validation evaluation. Each candidate is evaluated on a validation set of random seeds to obtain a coarse bi-objective performance estimate for anchor selection. Its validation performance is represented by the mean energy consumption and mean weighted tardiness over three independent runs. Let $S_i^{\mathrm{val}} = \widehat{\mathbb{E}}_i^{\mathrm{val}} + \widehat{\mathbb{T}}_i^{\mathrm{val}}$ denote the normalised validation score of individual i, where $\widehat{\mathbb{E}}_i^{\mathrm{val}}$ and $\widehat{\mathbb{T}}_i^{\mathrm{val}}$ are its min–max normalised validation energy and tardiness, respectively.

Decision-point sampling. A reference CCGP individual is selected from the validation-evaluated candidates by minimising S_i^{val}. The reference individual is then executed to collect representative resource-allocation decision points. At each sampled decision point, all feasible resources are recorded with their current terminal attributes.

Behavioural representation. For each candidate resource-selection rule, its behaviour is measured by replaying it on the sampled decision points. At decision point d, the rule scores all feasible resources and selects the resource with the best priority. The behaviour vector of g_i is

$$\mathbf{b}_i = (\pi_i(d_1), \pi_i(d_2), \ldots, \pi_i(d_M)), \tag{16}$$

where $\pi_i(d_m)$ is the resource index selected by g_i at sampled decision point d_m, and $M = 20$ is the number of sampled decision points. In implementation, $\pi_i(d_m)$ is encoded as a resource-selection indicator vector, so tied best resources are represented as the same selected set. Rules producing identical behaviour vectors are grouped together, and the representative minimising S_i^{val} is retained.

Anchor selection. If the number of behaviourally unique rules does not exceed the requested pool size, all representatives are retained. For the default maximum pool size $K_{\max} = 5$, the validation front otherwise supplies the minimum-energy endpoint, a low-energy neighbour, a balanced-knee rule, a low-tardiness neighbour, and the minimum-tardiness endpoint. The endpoints minimise $\widehat{\mathbb{E}}_i^{\mathrm{val}}$ and $\widehat{\mathbb{T}}_i^{\mathrm{val}}$, respectively. After excluding the endpoints, each neighbour is drawn from the corresponding 5% normalised objective band, i.e., its corresponding normalised value does not exceed 0.05; leading candidates under that objective ranking are also considered when the band is sparse. Let i_E and i_T denote the two endpoint indices. The balanced anchor is

$$\begin{aligned} S_i^{\mathrm{bal}} &= \max\left\{\widehat{\mathbb{E}}_i^{\mathrm{val}}, \widehat{\mathbb{T}}_i^{\mathrm{val}}\right\} + 0.05\left(\widehat{\mathbb{E}}_i^{\mathrm{val}} + \widehat{\mathbb{T}}_i^{\mathrm{val}}\right), \\ i_{\mathrm{bal}} &= \arg\min_{i \notin \{i_E, i_T\}} S_i^{\mathrm{bal}}. \end{aligned} \tag{17}$$

The five anchors are considered in the stated order. If an anchor candidate duplicates an already selected behaviour, the next behaviourally distinct candidate from the same region or objective ranking is used.

TABLE I
TERMINAL, FUNCTION, AND STATE-ENCODER ATTRIBUTES USED BY PRS-GPDRL.

Notation	Description	GP	DDQN
CAT	Category of a task.	T	T
SD	Sub-deadline of a task.	T	T
ET	Execution time of a task on a reference resource.	T	T
CPUA	Minimum CPU requirement of a task.	T	T
WT	Weight of a workflow.	T	T
UR	Upward rank of a task.	T	T
NSA	Number of direct successors of a task.	T	T
NPA	Number of direct predecessors of a task.	T	–
NKQ	Number of tasks in the ready queue.	T	–
TETQ	Total ET of tasks in the ready queue.	T	–
NRA	Number of remaining tasks in a workflow.	T	T
TETRA	Total ET of remaining workflow tasks.	T	T
VCT	Average communication time for a task.	T	T
NEW	Whether the candidate resource is newly created.	–	R
WAIT	Waiting time before the task starts on the candidate resource.	R	R
ETAR	Minimum execution time of a task on a resource.	R	R
CRCPU	Current remaining CPU of a resource.	R	R
ST	Slack time of a task on a resource.	R	R
STPR	Static power of a resource.	R	R
DPR	Dynamic power of a candidate resource.	R	R
NTR	Number of tasks on a resource.	R	R
NKW	Number of scheduled tasks from the same workflow in the Cloud or Edge tier.	R	R
CPUR	CPU capacity of a resource.	R	R
CTR	Communication time of a task on a resource.	R	R
w_T	Preference weight for weighted tardiness.	–	P
w_E	Preference weight for energy consumption.	–	P
Function set	$+$, $-$, $\times$, protected /, Max, and Min.	T/R	–

In the GP column, T and R denote task- and resource-selection rule terminals. In the DDQN column, T, R, and P denote task, candidate-resource, and preference attributes, respectively.

The retained resource-selection rules form the compact GP rule pool $\mathcal{G} = \{g_1, \ldots, g_K\}$. Combining $\mathcal{G}$ with the two modes gives the rule-mode action space $\mathcal{A}$.

C. Risk-Stratified Action Augmentation

The extracted GP rule pool provides complementary global resource-scoring behaviours, and the DDQN selects among their complete GP-induced priority rankings at rule level. PRS-GPDRL augments each rule with *base* and *risk* modes. The selected resource-selection rule provides the base priorities, while the risk mode adds a bounded, preference-conditioned local correction. The resulting action space combines rule-level behaviour selection with local preference adaptation over feasible resources.

Let $\mathbf{w} = (w_T, w_E)$ be the current preference vector, where w_T and w_E are the weights for weighted tardiness and energy consumption, respectively, and $w_T + w_E = 1$. At a resource-selection decision point, let $\mathcal{R}_t$ be the set of feasible resources and let $n = |\mathcal{R}_t|$. For a selected resource-selection rule $g_i \in \mathcal{G}$, let $\hat{g}_i(r)$ denote its min–max normalised priority score for resource $r \in \mathcal{R}_t$. For all min–max normalisations in this work, if all resources have the same value, the corresponding normalised score is set to zero for all resources.

PRS-GPDRL then estimates two objective-specific local costs for each resource in the same feasible-resource set. For the tardiness-oriented local cost, let α_t be the current ready task at decision time t, let $\mathsf{SD}(\alpha_t)$ be its sub-deadline, and let ω be the workflow containing α_t. For resource r, let $t^{\mathrm{ava}}_{\alpha_t,r}$ be the actual available time for assigning α_t to r. An allocation snapshot is computed by inserting α_t into r at $t^{\mathrm{ava}}_{\alpha_t,r}$ and applying the CPU allocation rule in Eq. (13). Let $\hat{x}^f_{\alpha_t,r}$ be the estimated finish time of α_t in this snapshot. The tardiness-oriented local cost combines the estimated time from the current decision point to task completion with the workflow-weighted tardiness relative to the sub-deadline:

$$c_T(r) = \hat{x}^f_{\alpha_t,r} - t + \rho^w(\omega)\max\{0, \hat{x}^f_{\alpha_t,r} - \mathsf{SD}(\alpha_t)\}, \quad (18)$$

For the energy-oriented local cost, PRS-GPDRL compares the resource state before and after assigning the current task. Let U^0_r and U^1_r be its CPU utilisation before and after assignment, and let H^0_r and H^1_r be the corresponding busy horizons measured from $t^{\mathrm{ava}}_{\alpha_t,r}$ to its tail completion time. Using the power terms defined in Section III, the dynamic energy increment is

$$\Delta E^{\mathrm{dyn}}_r = p^{dc}(r)\left((U^1_r)^3 H^1_r - (U^0_r)^3 H^0_r\right). \quad (19)$$

$\Delta E^{\mathrm{dyn}}_r$ is a lightweight estimate of the dynamic-energy increment caused by assigning the current task to resource r. The static energy increment depends on whether the resource is newly created, already active, or idle:

$$\Delta E^{\mathrm{sta}}_r = \begin{cases} p^s(r) H^1_r, & \text{new}, \\ p^s(r)\left(t^{\mathrm{tail},1}_r - t^{\mathrm{tail},0}_r\right), & \text{active}, \\ p^s(r)\left(t^{\mathrm{ava}}_{\alpha_t,r} - t^{\mathrm{last}}_r + H^1_r\right), & \text{idle}, \end{cases} \quad (20)$$

where $t^{\mathrm{tail},0}_r$ and $t^{\mathrm{tail},1}_r$ are the tail completion times before and after assignment, and t^{last}_r is the last utilisation event time of r. For an idle resource, the static-energy increment includes the idle interval since its last utilisation event and the new busy horizon caused by assigning the current task. The energy-oriented local cost is

$$c_E(r) = \Delta E^{\mathrm{dyn}}_r + \Delta E^{\mathrm{sta}}_r. \quad (21)$$

Let $\hat{c}_T(r)$ and $\hat{c}_E(r)$ be the min–max normalised objective-specific local costs used to make tardiness and energy comparable within the current feasible-resource set. A preference-weighted auxiliary score is then defined as

$$c_{\mathbf{w}}(r) = w_T \hat{c}_T(r) + w_E \hat{c}_E(r). \quad (22)$$

The auxiliary score ranks the feasible resources for risk stratification. Let $\mathrm{rank}_{\mathbf{w}}(r) \in \{1, \ldots, n\}$ be the ascending rank of $c_{\mathbf{w}}(r)$. PRS-GPDRL maps the rank to four risk strata:

$$\chi_{\mathbf{w}}(r) = \begin{cases} 0.00, & \mathrm{rank}_{\mathbf{w}}(r) \le \lceil 0.10n \rceil, \\ 0.20, & \lceil 0.10n \rceil < \mathrm{rank}_{\mathbf{w}}(r) \le \lceil 0.25n \rceil, \\ 0.50, & \lceil 0.25n \rceil < \mathrm{rank}_{\mathbf{w}}(r) \le \lceil 0.50n \rceil, \\ 1.00, & \mathrm{rank}_{\mathbf{w}}(r) > \lceil 0.50n \rceil. \end{cases} \quad (23)$$

Risk denotes the relative undesirability indicated by the auxiliary ranking. A lower risk level indicates that the resource is ranked closer to the preferred local choice under the requested preference. The cutoffs define top, near-top, medium-risk, and high-risk resource groups, respectively.

For the base mode, the risk term is disabled. For the risk mode, the mixing coefficient is

$$\beta_{\mathrm{risk}}(\mathbf{w}) = \beta_{\max}|w_T - w_E|^2, \quad (24)$$

Here, $|w_T - w_E|$ measures the departure from the balanced preference. When $w_T = w_E = 0.5$, the risk correction is disabled and the risk mode reduces to the base mode. Its influence increases smoothly towards either objective endpoint, reaching $\beta_{\max}$ at an extreme preference.

The final priority score of the resource-selection rule g_i under mode m is

$$p_m(r; g_i) = \begin{cases} \hat{g}_i(r), & m = \text{base}, \\ \hat{g}_i(r) + \beta_{\mathrm{risk}}(\mathbf{w})\chi_{\mathbf{w}}(r), & m = \text{risk}. \end{cases} \quad (25)$$

For a selected rule g^* and mode m^*, the selected resource is $r^*_t = \arg\min_{r \in \mathcal{R}_t} p_{m^*}(r; g^*)$.

D. MDP Formulation for Preference-Aware Rule Selection

The resource-selection problem is formulated as an MDP over task-ready decision points. It is represented by $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathbf{c}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the rule-mode action space, $\mathcal{P}$ is the state-transition kernel, $\mathbf{c}$ is the vector cost, and γ is the discount factor.

At decision point t, state $s_t \in \mathcal{S}$ contains the task, candidate-resource, and preference attributes marked in the DDQN column of Table I. Task and preference features are fixed-length, whereas the variable-size candidate-resource list is padded and accompanied by a validity mask.

Let $\mathcal{M} = \{\text{base}, \text{risk}\}$ be the mode set. The action space is

$$\mathcal{A} = \mathcal{G} \times \mathcal{M}, \qquad a_t = (g_i, m), \qquad g_i \in \mathcal{G},\ m \in \mathcal{M}. \quad (26)$$

Retaining K resource-selection rules therefore yields $2K$ rule-mode actions. Executing a_t applies the selected rule and mode to the feasible resources as defined in Section IV-C and assigns the current task to r_t^*. The scheduling process then advances to the next task-ready decision point or episode termination, producing the next state and transition cost.

Each MDP transition supplies the two-dimensional immediate cost used to train the DDQN. Let ΔT_t and ΔE_t denote the tardiness-related cost and energy increment, respectively, produced by the simulator transition from one decision point to the next. The stored cost is

$$\mathbf{c}_t = (\Delta T_t, \kappa \Delta E_t), \tag{27}$$

where κ aligns the numerical scales of the two components during training; evaluation uses the original objective values.

Algorithm 1: Online preference-conditioned resource selection

Input: State s_t containing preference $\mathbf{w}$; ready task α_t; feasible resource set $\mathcal{R}_t$
Output: Selected resource r_t^*

Encode s_t and evaluate $\mathbf{Q}_\theta(s_t, a)$ for all $a \in \mathcal{A}$;
Select $a_t^* = (g^*, m^*)$ by Eqs. (30) and (31);
Evaluate and normalise $g^*(r)$ over all $r \in \mathcal{R}_t$;
if $m^* = \text{base}$ **then** Return $r_t^* = \arg\min_{r \in \mathcal{R}_t} \hat{g}^*(r)$;
Estimate the normalised costs $\hat{c}_T(r)$ and $\hat{c}_E(r)$ over $\mathcal{R}_t$;
Construct the auxiliary score $c_{\mathbf{w}}(r)$ and the four-level $\chi_{\mathbf{w}}(r)$ by Eqs. (22) and (23);
Compute $\beta_{\text{risk}}(\mathbf{w})$ by Eq. (24);
Return $r_t^* = \arg\min_{r \in \mathcal{R}_t} p_{\text{risk}}(r; g^*)$ using Eq. (25);

E. Preference-Conditioned Multi-Objective DDQN

PRS-GPDRL uses a preference-conditioned multi-objective DDQN [41] to select rule-mode actions. At each decision point, the observation is first encoded into a preference-conditioned representation and then fed to a vector-valued DDQN. The resulting two-objective action-cost estimates are scalarised by the current preference for action selection and DDQN updates.

Fig. 2 illustrates the preference-conditioned state encoder and rule-mode action selection. The raw observation s_t contains task features, candidate-resource features, and preference weights. The state encoder maps s_t to embedding $\mathbf{z}_t$, and the DDQN estimates vector action costs over $\mathcal{A}$. Augmented weighted Tchebycheff scalarisation selects $a_t^* = (g^*, m^*)$, where $g^* \in \mathcal{G}$ and $m^* \in \mathcal{M}$ are the selected rule and mode. Algorithm 1 applies them to score the feasible resources and assign ready task α_t to r_t^*.

Let $\mathbf{x}_t^{\text{task}}$, $\mathbf{X}_t^{\text{res}}$, and $\mathbf{w}$ denote the task features, candidate-resource features, and preference vector, respectively. Continuous task and resource attributes are transformed using signed logarithmic scaling, while categorical indicators remain unchanged. The task encoder comprises two 64-unit layers, each followed by an ELU activation. Feasible resources are separated into Edge Node and Cloud Server sets. Each tier-specific resource encoder contains two 64-unit ELU layers that project resource features into key representations, followed by a 64-dimensional linear value projection. Single-head masked scaled dot-product attention uses the task embedding as the query and the projected resource representations as keys to aggregate the corresponding values into a 64-dimensional tier-level representation. The task embedding and the two tier-level representations are concatenated and passed through two 128-unit fusion layers, with an ELU activation after the first, to produce $\mathbf{h}_{\text{fuse}} \in \mathbb{R}^{128}$.

A feature-wise linear modulation (FiLM) layer [42] conditions the fused representation on the preference. A two-layer conditioning MLP maps the two preference weights through a 32-unit ELU hidden layer to a 256-dimensional output, which is split into 128-dimensional raw-scale and shift vectors. The scale is parameterised as $\boldsymbol{\gamma}_{\text{film}}(\mathbf{w}) = 1 + 0.3 \tanh(\widetilde{\boldsymbol{\gamma}}_{\text{film}}(\mathbf{w}))$, which bounds its elements between 0.7 and 1.3 and prevents multiplicative sign reversal or excessive amplification; the shift remains unconstrained. The FiLM transformation is

$$\mathbf{z}_t = \boldsymbol{\gamma}_{\text{film}}(\mathbf{w}) \odot \mathbf{h}_{\text{fuse}} + \boldsymbol{\delta}_{\text{film}}(\mathbf{w}), \tag{28}$$

where $\boldsymbol{\gamma}_{\text{film}}(\mathbf{w})$ and $\boldsymbol{\delta}_{\text{film}}(\mathbf{w})$ are preference-dependent scale and shift vectors. The resulting $\mathbf{z}_t \in \mathbb{R}^{128}$ is the DDQN state embedding.

The 128-dimensional state embedding is processed by a shared two-layer MLP with hidden sizes 256 and 128 and ReLU activations. A linear output layer then produces a two-dimensional cost estimate for every rule-mode action:

$$\mathbf{Q}_\theta(s_t, a) = \left(Q_\theta^T(s_t, a), Q_\theta^E(s_t, a)\right), \quad a \in \mathcal{A}. \tag{29}$$

Here, Q_θ^T and Q_θ^E estimate cumulative tardiness and energy costs, respectively, preserving both dimensions for preference-dependent scalarisation.

Since lower values are better for both objectives, action selection minimises an augmented weighted Tchebycheff cost. For state s, the estimated ideal component for objective j is $q_\theta^{j,\min}(s) = \min_{a' \in \mathcal{A}} Q_\theta^j(s, a')$, where $j \in \{T, E\}$. The scalarised cost is

$$\begin{aligned}\psi_{\mathbf{w}}(\mathbf{Q}_\theta(s,a)) = &\max_{j \in \{T,E\}} w_j \Big(Q_\theta^j(s,a) - q_\theta^{j,\min}(s)\Big) \\ &+ \zeta \sum_{j \in \{T,E\}} w_j \Big(Q_\theta^j(s,a) - q_\theta^{j,\min}(s)\Big),\end{aligned} \tag{30}$$

where ζ is the augmentation coefficient. The estimated ideal point gives objective-wise deviations from the best predicted action costs, and the augmentation term reduces ties. The greedy action is

$$a_t^* = \arg\min_{a \in \mathcal{A}} \psi_{\mathbf{w}}\left(\mathbf{Q}_\theta(s_t, a)\right). \tag{31}$$

The preference-conditioned vector action-cost function is trained using Double DQN [41] with experience replay and a periodically updated target network. At each episode, a preference vector is sampled and included in the state. The remaining training parameters are reported in Section V-C.

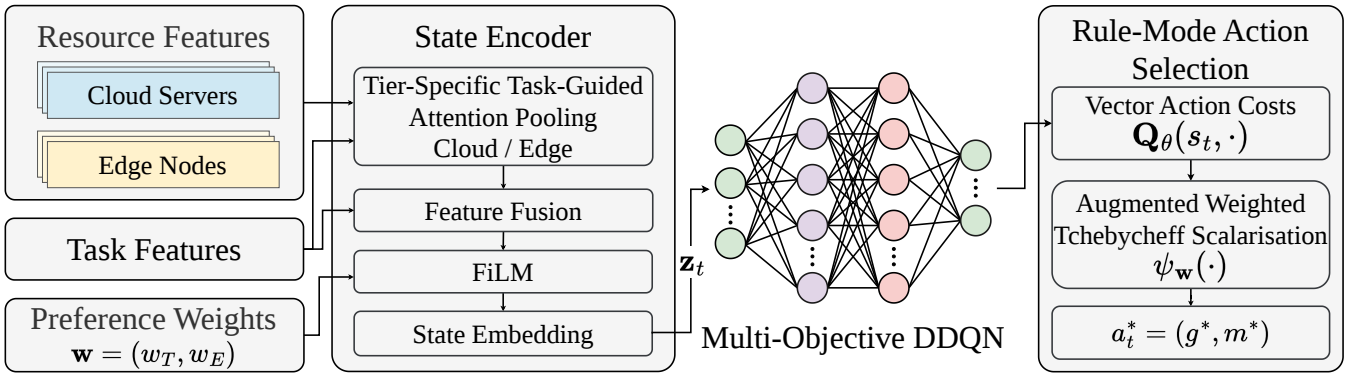

Fig. 2. Preference-conditioned multi-objective DDQN for rule-mode action selection in PRS-GPDRL.

F. Training and Inference Procedure

After CCGP training and action extraction in Section IV-B, the action space is fixed for offline DRL training. A scenario is denoted by $\langle \mathrm{wfNum}, \lambda \rangle$, where wfNum is the number of submitted workflows and λ is their arrival rate. Training and testing are performed separately for each scenario using the fixed action space. Each training episode samples a preference, includes it in the state, and learns from vector-cost transitions at ready-task decision points. The trained DDQN and fixed rule-mode action space constitute the PRS-GPDRL agent.

During inference, the PRS-GPDRL agent uses the DDQN to select a rule-mode action and assigns α_t to the feasible resource with the lowest corresponding priority. Scheduling then proceeds to the next task-ready decision point.

V. Performance Evaluation

A. Compared Algorithms

PRS-GPDRL is compared with CCGP, GATES, and LWFF.

- **CCGP** [17] coevolves task-selection, resource-selection, and container-scaling GP rules to construct non-dominated scheduling heuristics. For the problem model in Section III, our implementation retains its cooperative task- and resource-rule evolution but omits the container-scaling subpopulation because container CPU is assigned by Eq. (13). Each combined heuristic is evaluated on energy and weighted tardiness, and the final non-dominated heuristics form the approximation set.
- **GATES** [19] uses graph attention to encode ready tasks, workflow dependencies, and candidate resources, and evolves a neural resource-selection policy through an evolution strategy. Our implementation retains this graph representation and direct resource-selection policy. To support the two objectives, policy networks are evaluated on energy and weighted tardiness, NSGA-II performs individual selection, and offspring networks are generated by Gaussian perturbations of the selected network parameters. The final non-dominated policies form the approximation set.
- **LWFF** [33] is a constructive heuristic that selects resources from non-dominated candidates according to resource waste and completion speed. In our implementation, ready tasks are ordered by upward rank, and the energy and tardiness estimates of feasible resources are min–max normalised and combined under each requested preference. Expected finish time resolves ties. Applying LWFF over the evaluation preference grid and retaining the non-dominated schedules produces its approximation set.

B. Cloud–Edge Environment and Workloads

Experiments use the event-driven workflow simulator introduced in [17]. Its container-deployment decision is replaced with the deterministic CPU allocation rule in Eq. (13), with allocations updated at the task events specified in Section III. All methods use the same workflow instances, feasible resource set, communication model, CPU allocation rule, and energy accounting.

Table II lists five representative resource types for each tier. The configurations are derived from AWS EC2[1], and the power parameters follow the Teads model[2]. The Edge tier contains 19 fixed Edge Nodes, whereas Cloud Servers of the listed types can be provisioned on demand. Each task incurs a 50-ms container initialisation delay [6], and provisioning a new Cloud Server takes 55.9 s. Following [43], [44], LAN and WAN bandwidths are set to 2.0 and 0.5 GB/s, and their communication latencies are 0.5 and 30 ms, respectively.

Workflow templates are sampled from Alibaba Cluster-trace-v2018 [3]. The public trace supplies task dependencies and execution information, from which the DAG workflows used by the simulator are reconstructed. Task input and output data sizes follow a discrete uniform distribution between 500 and 5000 MB. Each task is independently assigned as CPU-sensitive or CPU-insensitive with equal probability. Workflow weights are independently sampled from $\{1, 2, 4\}$ with probabilities 0.2, 0.6, and 0.2, respectively, following [45].

Workflow arrivals follow a Poisson process with rate λ, so the inter-arrival time is exponentially distributed with mean $1/\lambda$ [46]. Following [17], reference execution and Edge transfer times are propagated along each DAG. Let L_ω be the

[1]https://aws.amazon.com/ec2/

[2]https://engineering.teads.com/sustainability/carbon-footprint-estimator-for-aws-instances/

[3]https://github.com/alibaba/clusterdata

TABLE II
CONFIGURATIONS OF EDGE NODES AND CLOUD SERVERS[1,2].

Edge Nodes					Cloud Servers				
Type (number)	ECU	CPU idle power (W)	CPU full power (W)	Base power (W)	Type	ECU	CPU idle power (W)	CPU full power (W)	Base power (W)
m1.small (3)	1	0.72	5.76	1.2	m4.xlarge	13	1.94	15.62	3.2
m3.medium (3)	3	0.87	6.97	1.4	m4.2xlarge	26	3.89	31.24	6.4
m3.large (4)	6.5	1.73	13.94	2.9	m4.4xlarge	53.5	7.77	62.47	12.9
m3.xlarge (4)	13	3.47	27.87	5.8	m4.10xlarge	124.5	24.12	193.88	40.0
m3.2xlarge (5)	26	6.94	55.74	11.5	m4.16xlarge	188	31.09	249.90	51.6

TABLE III
THE PARAMETER SETTINGS OF CCGP.

Parameter	Value
Population size	2 subpopulations of 50
Number of generations	51
Method for initialising population	Ramped-half-and-half
Initial minimum/maximum depth	2/7
Elitism	5
Maximal depth	8
Crossover/mutation/reproduction rate	0.80/0.15/0.05
Terminal/non-terminal selection rate	10%/90%
Environmental selection	NSGA-II

TABLE IV
THE PARAMETER CONFIGURATION OF DDQN AND RISK-STRATIFIED ACTION AUGMENTATION.

Parameter	Value
Training-step budget	1,000,000
Exploration rate ϵ	1.0 decays to 0.01
Discount factor γ	1.0
Learning rate	5×10^{-5}
Optimizer	Adam
Mini-batch size	256
Replay memory size	200,000
Target-update frequency	5,000
DDQN hidden-layer sizes	256/128
Maximum resource-selection rule-pool size $K_{\max}$	5
Maximum risk mixing $\beta_{\max}$	0.75
Energy-cost scaling κ	4.7×10^5
Tchebycheff augmentation ζ	0.05

largest earliest finish time obtained by this propagation. The workflow deadline is $\rho^d(\omega) = \rho^a(\omega) + \delta_\omega L_\omega$, where the deadline factor δ_ω is sampled uniformly from $\{0.8, 1.0, 1.5, 1.8\}$.

C. Training and Test Protocol

The main evaluation covers the 12 combinations of $\mathrm{wfNum} \in \{10, 15, 20, 30, 50, 100\}$ and $\lambda \in \{0.2, 0.8\}$. For each main-evaluation scenario, CCGP, GATES, and PRS-GPDRL are independently trained in 30 independent runs. In each run, PRS-GPDRL uses the GP rule pool extracted from the corresponding CCGP run. Each trained run is evaluated on the same 30 unseen test instances. Objective values are first averaged over these test instances within each run, and the means and standard deviations are then computed over the 30 independent runs. For each test instance, all methods use identical workflow arrivals, DAGs, task attributes, and deadlines.

The parameter settings of CCGP follow [17] and are presented in Table III. The DDQN and risk-stratified action augmentation configuration is detailed in Table IV. GATES uses a population of 100 policy networks and runs for 51 generations; it uses the bi-objective NSGA-II selection described in Section V-A.

The maximum requested resource-selection rule-pool size is $K_{\max} = 5$. Behavioural filtering may leave fewer than five unique rules in a particular run; the actual pool size in run r is denoted by $K_r \le K_{\max}$. Each retained rule is paired with the two modes, so that run r has $2K_r$ nominal rule-mode actions. The DDQN is trained with sampled preferences and evaluated on

$$w_T \in \{0, 0.05, \ldots, 0.95, 1\}, \qquad w_E = 1 - w_T. \tag{32}$$

Checkpoints are selected according to validation performance over these evaluation preferences.

D. Evaluation Metrics

For each run, the non-dominated solutions generated over the preference grid form an approximation set. Solution quality is assessed by hypervolume (HV) and inverted generational distance (IGD), which measure complementary aspects of convergence and distribution. Before calculating either metric, each objective $f_j \in \{\mathbb{T}, \mathbb{E}\}$ is normalised within a scenario as

$$\widehat{f}_j(x) = \frac{f_j(x) - f_j^{\min}}{f_j^{\max} - f_j^{\min}}, \tag{33}$$

where $f_j^{\min}$ and $f_j^{\max}$ are obtained from the pooled solutions of all compared methods in that scenario. If the denominator is zero, the normalised value is set to zero.

HV is the dominated area between an approximation set and the reference point $(1, 1)$ in the normalised objective space. A larger HV indicates a set with better convergence, coverage, or both. Because the true Pareto front is unknown, the IGD reference set is constructed by merging the solutions of all methods and retaining the globally non-dominated solutions for the scenario. For an approximation set P and reference set Q, IGD is

$$\mathrm{IGD}(P, Q) = \frac{1}{|Q|} \sum_{q \in Q} \min_{p \in P} \|p - q\|_2. \tag{34}$$

A smaller IGD indicates that the approximation set is closer to and more representative of the pooled reference front.

E. Overall Performance

Tables V and VI report the mean and standard deviation over the 30 independent runs. For each scenario and metric, overall

TABLE V
THE MEAN AND STANDARD DEVIATION OF THE HV VALUES OVER THE 30 INDEPENDENT RUNS OF THE COMPARED ALGORITHMS.

$\langle \text{wfNum}, \lambda \rangle$	GATES	LWFF	CCGP	PRS-GPDRL
$\langle 10, 0.2 \rangle$	0.525±0.196	0.590±0.138(≈)	0.772±0.105(↑)(↑)	0.859±0.072(↑)(↑)(↑)
$\langle 10, 0.8 \rangle$	0.538±0.217	0.619±0.129(↑)	0.782±0.125(↑)(↑)	0.882±0.068(↑)(↑)(↑)
$\langle 15, 0.2 \rangle$	0.564±0.128	0.601±0.109(≈)	0.800±0.116(↑)(↑)	0.862±0.089(↑)(↑)(↑)
$\langle 15, 0.8 \rangle$	0.536±0.134	0.649±0.113(↑)	0.747±0.154(↑)(↑)	0.843±0.083(↑)(↑)(↑)
$\langle 20, 0.2 \rangle$	0.604±0.172	0.630±0.125(≈)	0.786±0.140(↑)(↑)	0.868±0.091(↑)(↑)(↑)
$\langle 20, 0.8 \rangle$	0.530±0.173	0.646±0.109(↑)	0.776±0.112(↑)(↑)	0.825±0.061(↑)(↑)(↑)
$\langle 30, 0.2 \rangle$	0.635±0.143	0.674±0.096(↑)	0.815±0.108(↑)(↑)	0.864±0.065(↑)(↑)(↑)
$\langle 30, 0.8 \rangle$	0.570±0.139	0.699±0.097(↑)	0.775±0.100(↑)(↑)	0.797±0.099(↑)(↑)(≈)
$\langle 50, 0.2 \rangle$	0.610±0.152	0.701±0.098(↑)	0.807±0.105(↑)(↑)	0.851±0.116(↑)(↑)(↑)
$\langle 50, 0.8 \rangle$	0.643±0.167	0.749±0.105(↑)	0.782±0.138(↑)(↑)	0.814±0.113(↑)(↑)(↑)
$\langle 100, 0.2 \rangle$	0.576±0.200	0.697±0.098(↑)	0.811±0.099(↑)(↑)	0.845±0.082(↑)(↑)(↑)
$\langle 100, 0.8 \rangle$	0.787±0.093	0.861±0.065(↑)	0.842±0.078(↑)(↓)	0.804±0.090(≈)(↓)(↓)
W/D/L	0/1/11	1/0/11	1/1/10	N/A
AverageRank	3.597	3.008	1.992	1.403

TABLE VI
THE MEAN AND STANDARD DEVIATION OF THE IGD VALUES OVER THE 30 INDEPENDENT RUNS OF THE COMPARED ALGORITHMS.

$\langle \text{wfNum}, \lambda \rangle$	GATES	LWFF	CCGP	PRS-GPDRL
$\langle 10, 0.2 \rangle$	0.317±0.138	0.257±0.078(↑)	0.104±0.040(↑)(↑)	0.052±0.038(↑)(↑)(↑)
$\langle 10, 0.8 \rangle$	0.304±0.146	0.217±0.069(↑)	0.104±0.040(↑)(↑)	0.041±0.024(↑)(↑)(↑)
$\langle 15, 0.2 \rangle$	0.262±0.096	0.176±0.056(↑)	0.077±0.033(↑)(↑)	0.054±0.036(↑)(↑)(↑)
$\langle 15, 0.8 \rangle$	0.298±0.084	0.186±0.063(↑)	0.091±0.060(↑)(↑)	0.050±0.032(↑)(↑)(↑)
$\langle 20, 0.2 \rangle$	0.261±0.126	0.185±0.070(↑)	0.084±0.042(↑)(↑)	0.055±0.042(↑)(↑)(↑)
$\langle 20, 0.8 \rangle$	0.266±0.126	0.187±0.079(↑)	0.078±0.036(↑)(↑)	0.061±0.046(↑)(↑)(≈)
$\langle 30, 0.2 \rangle$	0.231±0.102	0.136±0.047(↑)	0.078±0.038(↑)(↑)	0.053±0.049(↑)(↑)(↑)
$\langle 30, 0.8 \rangle$	0.278±0.090	0.124±0.033(↑)	0.074±0.033(↑)(↑)	0.068±0.032(↑)(↑)(≈)
$\langle 50, 0.2 \rangle$	0.230±0.093	0.109±0.038(↑)	0.087±0.046(↑)(↑)	0.066±0.051(↑)(↑)(↑)
$\langle 50, 0.8 \rangle$	0.230±0.102	0.094±0.030(↑)	0.079±0.041(↑)(≈)	0.078±0.039(↑)(↑)(≈)
$\langle 100, 0.2 \rangle$	0.257±0.131	0.131±0.050(↑)	0.071±0.035(↑)(↑)	0.083±0.037(↑)(↑)(≈)
$\langle 100, 0.8 \rangle$	0.188±0.078	0.088±0.048(↑)	0.069±0.028(↑)(≈)	0.140±0.067(↑)(↓)(↓)
W/D/L	0/0/12	1/0/11	1/4/7	N/A
AverageRank	3.778	2.869	1.828	1.525

differences among the four algorithms are assessed using the Iman–Davenport correction of the Friedman test, with the matched runs as blocks. Pairwise significance symbols are obtained from two-sided paired Wilcoxon signed-rank tests on the matched run-level metric values at a significance level of 0.05. Within each result cell, ↑, ↓, and ≈ indicate statistically better, worse, and similar performance, respectively, against the methods in the preceding columns. The W/D/L row records each baseline's wins, draws, and losses against PRS-GPDRL. AverageRank averages the within-block ranks over all scenarios and runs, with a smaller value being preferable.

1) Comparison of Hypervolume: Table V shows that PRS-GPDRL obtains the best HV average rank of 1.403, ahead of CCGP (1.992), LWFF (3.008), and GATES (3.597). It achieves significantly higher HV than CCGP in ten scenarios and comparable HV in one of the remaining two. It also significantly exceeds GATES and LWFF in 11 scenarios each.

Across all ten scenarios with at most 50 workflows, PRS-GPDRL has the highest mean HV at both arrival rates. For example, in $\langle 10, 0.8 \rangle$, its mean HV is 0.882, compared with 0.782 for CCGP and 0.619 for LWFF. Since HV reflects both convergence and coverage, the consistent gains across these workload sizes indicate that PRS-GPDRL produces higher-quality and better-covered approximation sets.

2) Comparison of Inverted Generational Distance: Table VI shows that PRS-GPDRL also obtains the best IGD average rank of 1.525, followed by CCGP (1.828), LWFF (2.869), and GATES (3.778). Relative to CCGP, it achieves significantly lower IGD in seven scenarios and statistically similar IGD in four. PRS-GPDRL also significantly improves on GATES in all 12 scenarios and on LWFF in 11 scenarios.

PRS-GPDRL has the lowest mean IGD in all ten scenarios with at most 50 workflows. In $\langle 10, 0.8 \rangle$, its mean IGD is 0.041, compared with 0.104 for CCGP and 0.217 for LWFF. These results show that its approximation sets remain closer to and more representative of the pooled reference front across different workflow counts and arrival rates.

Taken together, the two metrics show that PRS-GPDRL is statistically better than or comparable to CCGP in 11 of the 12 scenarios for each metric. The common exception is $\langle 100, 0.8 \rangle$, where CCGP and LWFF obtain higher HV and lower IGD. Overall, the agreement between HV and IGD supports the effectiveness of PRS-GPDRL in constructing well-covered tardiness–energy trade-offs close to the pooled reference front.

VI. CONCLUSION

This paper studies preference-aware online scheduling of dynamic workflows in heterogeneous Cloud–Edge systems, with total weighted tardiness and energy as the two objectives. PRS-GPDRL connects evolutionary rule discovery with online reinforcement learning: CCGP evolves coupled scheduling rules, a diversity-aware procedure extracts a compact resource-selection rule pool, and a preference-conditioned multi-objective DDQN selects risk-stratified rule-mode actions at task-allocation decision points. The resulting design retains the efficient execution of GP priority rules while adapting their use to the current state and requested objective trade-off. Across the 12 scenarios and 30 independent runs, PRS-GPDRL obtains the best average ranks of 1.403 for HV and 1.525 for IGD. It is statistically better than or comparable to CCGP in 11 scenarios for each metric, demonstrating the value of state- and preference-dependent rule-mode selection for constructing tardiness–energy trade-offs.

REFERENCES

[1] International Data Corporation, "IDC estimates global spending on edge computing to grow at 13.8%, reaching nearly $380 billion by 2028," 2025, accessed: Aug. 10, 2026. [Online]. Available: https://www.idc.com/resource-center/press-releases/idc-estimates-global-spending-on-edge-computing-to-grow-at-13-8-reaching-nearly-380-billion-by-2028/

[2] International Energy Agency, "Energy and AI," 2025, accessed: Jul. 27, 2026. [Online]. Available: https://www.iea.org/reports/energy-and-ai

[3] S. Li, W. Wu, H. Zhang, Y. Liu, W. Lin, and K. Li, "Revisiting workflow scheduling with the power of edge computing: Taxonomy, review, and open challenges," *Comput. Sci. Rev.*, vol. 60, p. 100887, 2026.

[4] R. Bouabdallah and F. Fakhfakh, "Workflow scheduling in cloud–fog computing environments: A systematic literature review," *Concurr. Comput. Pract. Exp.*, vol. 36, no. 28, p. e8304, 2024.

[5] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, "WfCommons: A framework for enabling scientific workflow research and development," *Future Gener. Comput. Syst.*, vol. 128, pp. 16–27, 2022.

[6] R. Ranjan, I. S. Thakur, G. S. Aujla, N. Kumar, and A. Y. Zomaya, "Energy-efficient workflow scheduling using container-based virtualization in software-defined data centers," *IEEE Trans. Ind. Informat.*, vol. 16, no. 12, pp. 7646–7657, 2020.

[7] Kubernetes, "Resize CPU and memory resources assigned to containers," 2026, accessed: Jul. 27, 2026. [Online]. Available: https://kubernetes.io/docs/tasks/configure-pod-container/resize-container-resources/

[8] Docker Inc., "docker container update," accessed: Jul. 27, 2026. [Online]. Available: https://docs.docker.com/reference/cli/docker/container/update/

[9] ——, "Resource constraints," accessed: Jul. 27, 2026. [Online]. Available: https://docs.docker.com/engine/containers/resource_constraints/

[10] A. Jayanetti, S. Halgamuge, and R. Buyya, "Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge–cloud computing environments," *Future Gener. Comput. Syst.*, vol. 137, pp. 14–30, 2022.

[11] J. Zhang, Z. Ning, M. Waqas, H. Alasmary, S. Tu, and S. Chen, "Hybrid edge-cloud collaborator resource scheduling approach based on deep reinforcement learning and multiobjective optimization," *IEEE Trans. Comput.*, vol. 73, no. 1, pp. 192–205, 2024.

[12] K.-R. Escott, H. Ma, and G. Chen, "A genetic programming hyper-heuristic approach to design high-level heuristics for dynamic workflow scheduling in cloud," in *Proc. IEEE Symp. Ser. Comput. Intell.*, 2020, pp. 3141–3148.

[13] Y. Yu, T. Shi, H. Ma, and G. Chen, "A genetic programming-based hyper-heuristic approach for multi-objective dynamic workflow scheduling in cloud environment," in *Proc. IEEE Congr. Evol. Comput.*, 2022, pp. 1–8.

[14] M. Xu, Y. Mei, S. Zhu, B. Zhang, T. Xiang, F. Zhang, and M. Zhang, "Genetic programming for dynamic workflow scheduling in fog computing," *IEEE Trans. Serv. Comput.*, vol. 16, no. 4, pp. 2657–2671, 2023.

[15] Z. Sun, Y. Mei, F. Zhang, H. Huang, C. Gu, and M. Zhang, "Multi-tree genetic programming hyper-heuristic for dynamic flexible workflow scheduling in multi-clouds," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2687–2703, 2024.

[16] Y. Yang, G. Chen, H. Ma, S. Hartmann, and M. Zhang, "Dual-tree genetic programming with adaptive mutation for dynamic workflow scheduling in cloud computing," *IEEE Trans. Evol. Comput.*, vol. 29, no. 5, pp. 1692–1706, 2025.

[17] Z. Sun, F. Zhang, Y. Mei, H. Huang, C. Gu, B. Wang, and M. Zhang, "Cooperative coevolution genetic programming for dynamic joint workflow scheduling and container scaling in cloud-fog computing," *IEEE Trans. Serv. Comput.*, vol. 19, no. 1, pp. 225–239, 2026.

[18] B. Xiao, C. Yu, X. Chen, Z. Chen, and G. Min, "Multi-agent collaboration for workflow task offloading in end-edge-cloud environments using deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 36, no. 11, pp. 2281–2296, 2025.

[19] Y. Shen, G. Chen, H. Ma, and M. Zhang, "GATES: Cost-aware dynamic workflow scheduling via graph attention networks and evolution strategy," in *Proc. 34th Int. Joint Conf. Artif. Intell.*, 2025, pp. 8635–8643.

[20] R. Liu, R. Piplani, and C. Toro, "Deep reinforcement learning for dynamic scheduling of a flexible job shop," *Int. J. Prod. Res.*, vol. 60, no. 13, pp. 4049–4069, 2022.

[21] M. Xu, Y. Mei, F. Zhang, and M. Zhang, "Niching genetic programming to learn actions for deep reinforcement learning in dynamic flexible scheduling," *IEEE Trans. Evol. Comput.*, vol. 30, no. 1, pp. 61–75, 2026.

[22] F. Zhao, C. Zhao, L. Wang, and H. Sang, "A deep reinforcement learning framework assisted by genetic programming for dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, 2025.

[23] Y. He, Z. Xu, J. Lin, Y. Li, and S. Zhang, "Deep reinforcement learning with evolved actions for dynamic workflow scheduling in distributed fog computing," *Neurocomputing*, vol. 677, p. 133115, 2026.

[24] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, "A survey of multi-objective sequential decision-making," *J. Artif. Intell. Res.*, vol. 48, pp. 67–113, 2013.

[25] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Reymond, T. Verstraeten, L. M. Zintgraf, R. Dazeley, F. Heintz, E. Howley, A. A. Irissappane, P. Mannion, A. Nowé, G. Ramos, M. Restelli, P. Vamplew, and D. M. Roijers, "A practical guide to multi-objective reinforcement learning and planning," *Auton. Agents Multi-Agent Syst.*, vol. 36, no. 1, p. 26, 2022.

[26] A. Abels, D. M. Roijers, T. Lenaerts, A. Nowé, and D. Steckelmacher, "Dynamic weights in multi-objective deep reinforcement learning," in *Proc. 36th Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 11–20.

[27] R. Yang, X. Sun, and K. Narasimhan, "A generalized algorithm for multi-objective reinforcement learning and policy adaptation," in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.

[28] M. Xu, Y. Mei, F. Zhang, Y. S. Ong, and M. Zhang, "Pareto set learning through genetic programming for multiobjective dynamic scheduling," *IEEE Trans. Evol. Comput.*, vol. 30, no. 3, pp. 942–956, 2026.

[29] G. Fan, X. Chen, Z. Li, H. Yu, and Y. Zhang, "An energy-efficient dynamic scheduling method of deadline-constrained workflows in a cloud environment," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3089–3103, 2023.

[30] Z. Sun, H. Huang, Z. Li, and C. Gu, "Energy-efficient real-time multi-workflow scheduling in container-based cloud," *J. Comb. Optim.*, vol. 49, no. 2, pp. 1–21, 2025.

[31] K. Li, "Energy-constrained DAG scheduling on edge and cloud servers with overlapped communication and computation," *J. Grid Comput.*, vol. 22, no. 3, p. 60, 2024.

[32] J. Lou, Z. Tang, S. Zhang, W. Jia, W. Zhao, and J. Li, "Cost-effective scheduling for dependent tasks with tight deadline constraints in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 10, pp. 5829–5845, 2023.

[33] H. M. Fard, R. Prodan, and F. Wolf, "Dynamic multi-objective scheduling of microservices in the cloud," in *Proc. IEEE/ACM Int. Conf. Utility Cloud Comput.*, 2020, pp. 386–393.

[34] K. Van Moffaert and A. Nowé, "Multi-objective reinforcement learning using sets of Pareto dominating policies," *J. Mach. Learn. Res.*, vol. 15, pp. 3663–3692, 2014.

[35] S. Parisi, M. Pirotta, and M. Restelli, "Multi-objective reinforcement learning through continuous Pareto manifold approximation," *J. Artif. Intell. Res.*, vol. 57, pp. 187–227, 2016.

[36] A. Abdolmaleki, S. Huang, L. Hasenclever, M. Neunert, F. Song, M. Zambelli, M. Martins, N. Heess, R. Hadsell, and M. Riedmiller, "A distributional view on multi-objective policy optimization," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 119, 2020, pp. 11–22.

[37] M. Reymond, E. Bargiacchi, and A. Nowé, "Pareto conditioned networks," in *Proc. 21st Int. Conf. Auton. Agents Multiagent Syst.*, 2022, pp. 1110–1118.

[38] Y. Yang, T. Zhou, M. Pechenizkiy, and M. Fang, "Preference controllable reinforcement learning with advanced multi-objective optimization," in *Proc. 42nd Int. Conf. Mach. Learn.*, vol. 267, 2025, pp. 71 612–71 647.

[39] X. Lin, Z. Yang, X. Zhang, and Q. Zhang, "Pareto set learning for expensive multi-objective optimization," in *Adv. Neural Inf. Process. Syst.*, vol. 35, 2022.

[40] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, 2002.

[41] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.

[42] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual reasoning with a general conditioning layer," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3942–3951.

[43] M. Goudarzi, H. Wu, M. Palaniswami, and R. Buyya, "An application placement technique for concurrent IoT applications in edge and fog computing environments," *IEEE Trans. Mobile Comput.*, vol. 20, no. 4, pp. 1298–1311, 2021.

[44] S. Pallewatta, V. Kostakos, and R. Buyya, "QoS-aware placement of microservices-based IoT applications in fog computing environments," *Future Gener. Comput. Syst.*, vol. 131, pp. 121–136, 2022.

[45] T. Hildebrandt and J. Branke, "On using surrogates with genetic programming," *Evol. Comput.*, vol. 23, no. 3, pp. 343–367, 2015.

[46] S. Sahoo, B. Sahoo, and A. K. Turuk, "A learning automata-based scheduling for deadline sensitive task in the cloud," *IEEE Trans. Serv. Comput.*, vol. 14, no. 6, pp. 1662–1674, 2021.